Republic of the Philippines
CEBU TECHNOLOGICAL UNIVERSITY
ARGAO CAMPUS
Ed Kintanar Street, Lamacan, Argao, Cebu
Website: http://www.argao.ctu.edu.ph E-mail: cdargao@ctu.edu.ph
Phone No.: (032) 401-0737 local 1700

Cross Platform Script Development
Week 5
Information Technology Department

Name:	
Course:	**BSIT**
Year & Section:	
Date:	
Topic:	JavaScript Designs Patterns – Strategy Pattern

Notes:

The Strategy Pattern is a behavioral design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable. The client (context) can switch between strategies dynamically without changing its own code.

In simple terms:

Instead of writing many **if/else** conditions for different behaviors, you separate each behavior into its own strategy and choose which one to use at runtime.

Structure

1. **Strategy** → Defines a common interface for all algorithms.
2. **Concrete Strategies** → Different implementations of the algorithm.
3. **Context** → Uses a strategy object, but doesn't know how it works.

In the **Strategy Pattern**, the **Context** is the class/object that:

- Holds a **reference to a strategy** (algorithm).
- Delegates work to the chosen strategy instead of implementing the logic itself.
- Does not care *how* the strategy works, only that it follows the agreed interface.

Think of **Context** as the "manager" — it doesn't do the work itself, it just decides **which worker (strategy)** to call.

Real-Life Examples

1. Payment Methods (E-commerce)

- You might let users pay using **Credit Card, PayPal, or GCash**.
- Instead of if (method === "paypal") ... else if (method === "gcash") ..., each payment type is its own **strategy**.

2. Travel Route Planning (Google Maps, Waze)

- Strategies: **Car Route, Walking Route, Public Transport Route**.
- The user selects a travel mode, and the app switches strategy.

3. Authentication (Web Apps)

- Strategies: **Login via Email/Password, Google Login, Facebook Login**.
- Each login method is encapsulated as a strategy.

4. Sorting Algorithms

- For small datasets, use **Bubble Sort**.
- For large datasets, use **Quick Sort**.
- You can switch strategies dynamically depending on the dataset.

Advantages

- Eliminates complex if/else or switch logic.
- Easier to **extend** (just add a new strategy without modifying old ones).
- Promotes **loose coupling** (context doesn't know how the strategy works).
- Makes code **cleaner, testable, and maintainable**.

Disadvantages

- Adds extra **classes/objects** (can be overkill for simple tasks).
- Client must know which strategy to choose.
- Can increase code complexity if there are too many strategies.

When to Use Strategy Pattern

- When you have multiple ways to perform the same task.
- When you want to **change behavior at runtime**.
- When you want to **avoid long conditional statements**.
- When adding a new algorithm should **not affect existing code**.

Strategy Pattern Code

```
//Note
// Strategy Interface
class RouteStrategy {
  buildRoute(start, end) {
    throw new Error("buildRoute() must be implemented");
  }
}

// Concrete Strategies
class CarRoute extends RouteStrategy {
  buildRoute(start, end) {
    console.log(`Car route from ${start} to ${end}`);
  }
}

class WalkingRoute extends RouteStrategy {
  buildRoute(start, end) {
    console.log(`Walking route from ${start} to ${end}`);
  }
}

class PublicTransportRoute extends RouteStrategy {
  buildRoute(start, end) {
    console.log(`Public transport route from ${start} to ${end}`);
  }
}

// Context
class Navigator {
  setStrategy(strategy) {
    this.strategy = strategy;
  }

  buildRoute(start, end) {
    if (!this.strategy) throw new Error("Strategy not set!");
    this.strategy.buildRoute(start, end);
  }
}

// Usage
```

```
const navigator = new Navigator();

navigator.setStrategy(new CarRoute());
navigator.buildRoute("Cebu City", "Davao City");

navigator.setStrategy(new WalkingRoute());
navigator.buildRoute("Ayala Mall", "IT Park");

navigator.setStrategy(new PublicTransportRoute());
navigator.buildRoute("Mactan Airport", "SM Seaside");
```